

Case Study:

CMS Back Office Upgrade

One workflow for catalogue operations

Search and reuse product data



Add photos to the guided form



Process images and draft the summary



Review and publish

01

Business improvement

I built a private operations CMS over an existing retail database. It brought product search, stock checks, image processing and AI-assisted copy into one form-based workspace. Staff could prepare and publish catalogue items through a guided process without needing access to database administration tools.

- Typical product onboarding fell from approximately 15 minutes to around 3-an improvement of roughly 80% for a repeated operational task.
- Product data, image preparation, catalogue copy and availability checks moved into one flow, reducing tool switching and repeated manual entry.
- Automated variants and previews created a more consistent image pipeline across a large catalogue.
- Role-specific access gave staff the actions they needed without exposing the full database or storage administration interface.
- Visible processing states replaced waiting without feedback during external image operations.

The operational problem

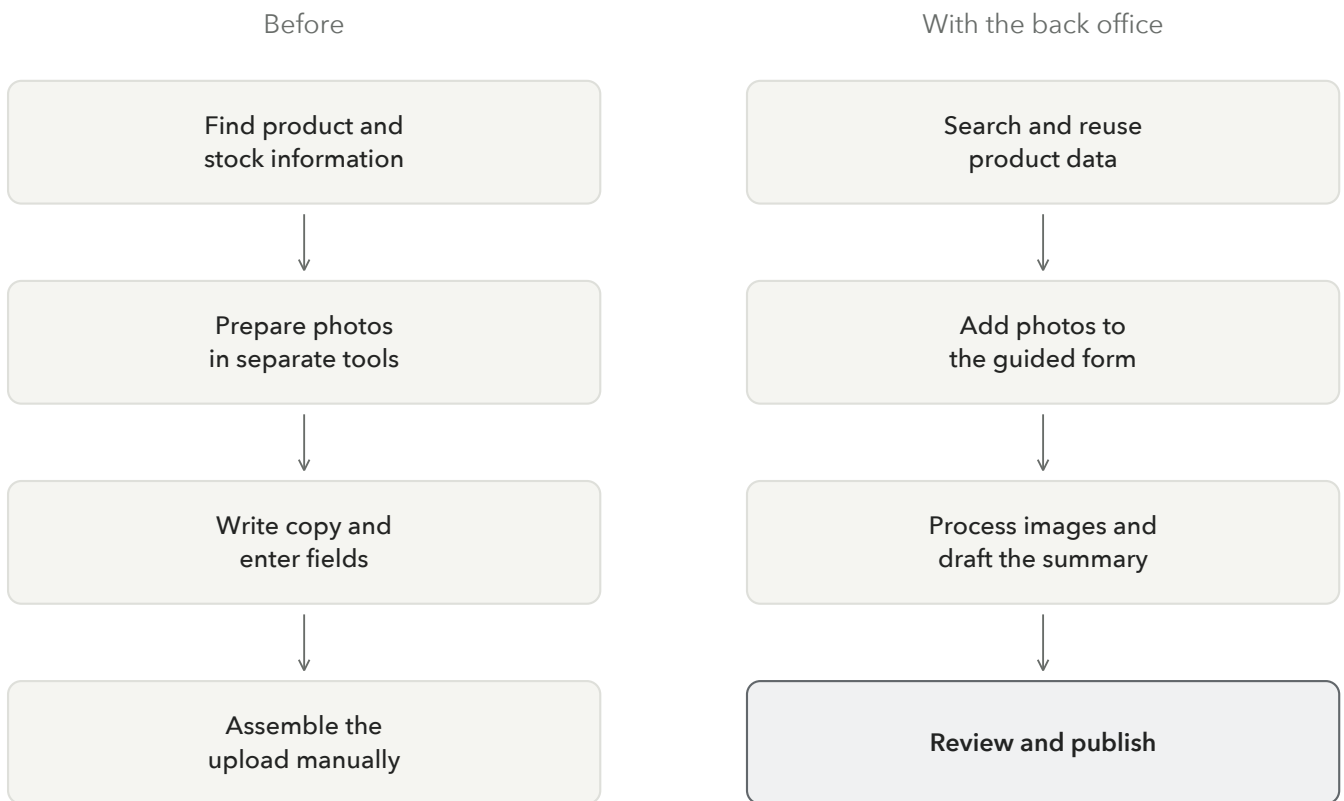
Adding a product involved searching external services, checking stock, preparing around ten photos, entering catalogue fields and writing a summary. Staff moved between tools before assembling everything in a large upload form. Direct Supabase access was too broad and too technical for this daily work. The team needed a back office that let operations staff complete routine tasks without relying on one technical person for every update.

Typical product onboarding

Before**About 15 minutes****With the back office****About 3 minutes**

02

A single workflow for staff



The CMS brings lookup, preparation and review into the same staff workflow.

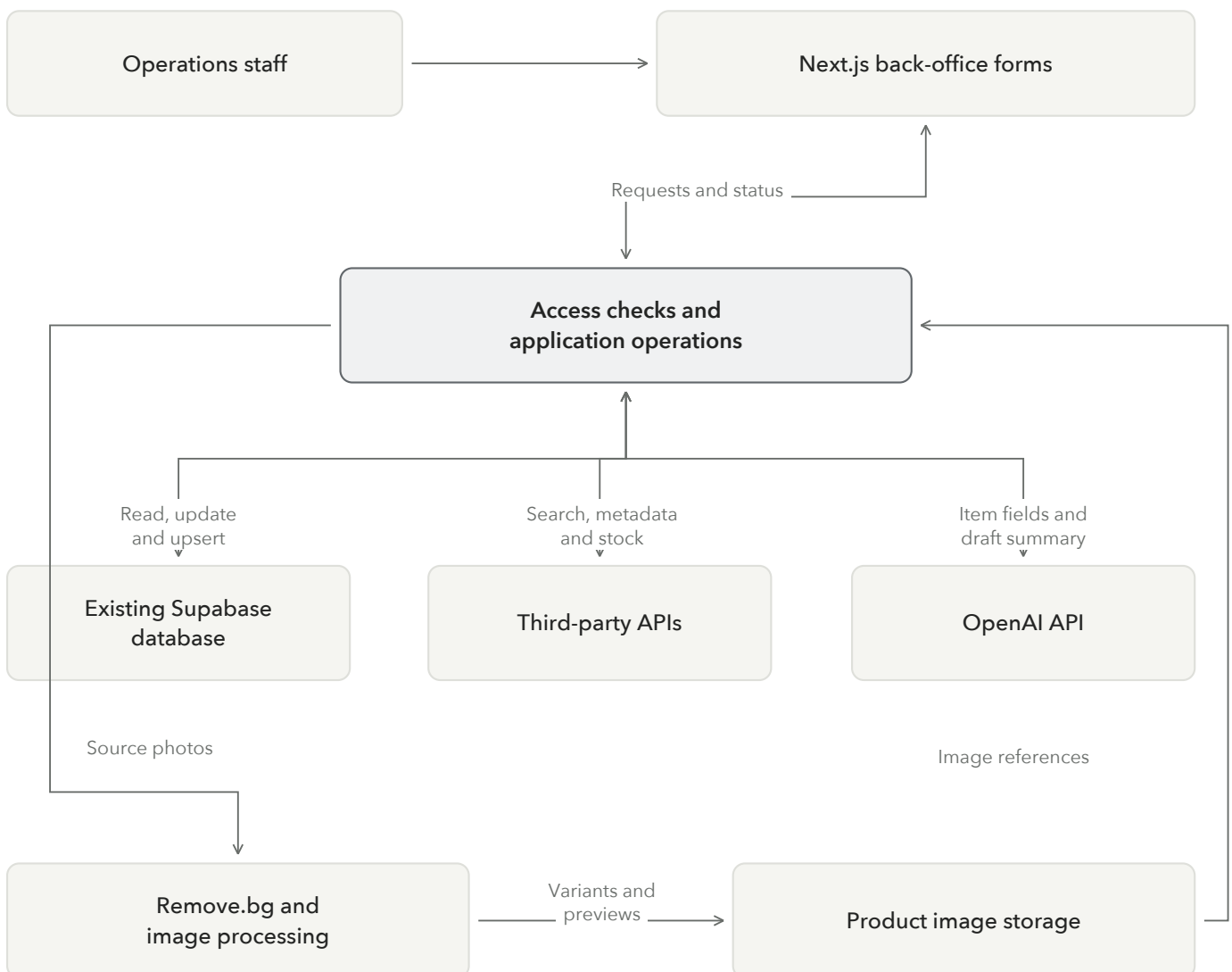
Search and autocomplete fill reusable product and colour information. Stock checks are available in the same workspace. Staff add the image set, follow processing status and review the assembled item before publication.

The application covers more than 15 screens with different forms. This gives staff a task-based interface for the catalogue while keeping database structure and third-party service details behind application operations.

Parts of the system

The Next.js application connects the staff interface to existing data and external services. The main responsibilities are:

- **Back-office forms:** product selection, data entry, image input, validation and review.
- **Application operations:** staff access checks, integration calls and controlled database updates.
- **Supabase database:** existing catalogue records and their relationships.
- **Search and stock APIs:** product matches, reusable metadata and availability.
- **Image pipeline:** background removal, image variants and previews.
- **OpenAI API:** draft summaries from the completed item fields.



The application joins external results into one item while the existing database remains the catalogue's source of truth.

The back office exposes the operations each staff role needs. Forms guide data entry, while access checks protect the underlying operations. Hiding database controls in the interface is only one part of that separation.

Data relationships and validation

Relationships without duplicate records

The database already contained tables and large datasets. The work started with those records and the relationships needed by the staff workflow. Rebuilding the catalogue or adding indexes broadly was not a practical starting point.

The data design focused on a few rules:

- Reuse existing record identities when search or selection finds a known item.
- Reference shared records instead of copying their fields into each form's result.
- Model many-to-many associations through relationships so the same record can be reused without creating another copy.
- Shape reads around the records and related fields a screen actually needs.

This made the database work part of product design. A choice in a form had to map to the right existing record and relationship, rather than become another piece of duplicated catalogue data.

Updates and upserts

Writes followed the intent of the staff action. Editing an existing item meant updating its relevant fields and relationships. Where a record could already exist, an upsert handled insertion or update around its identity.

An upsert still needs a clear way to identify the record. It does not decide whether two similar products are the same. Selection and relationship handling therefore mattered as much as the write itself.

The goal was to preserve existing data while making repeated edits predictable. That required deliberate updates, rather than treating every form submission as a request to create a new set of records.

Database types and validation

TypeScript types were generated from the Supabase schema and used in the data layer and form models. They helped keep database fields, query results and application code aligned as features were added.

Zod handled validation of entered values. The two layers served different jobs:

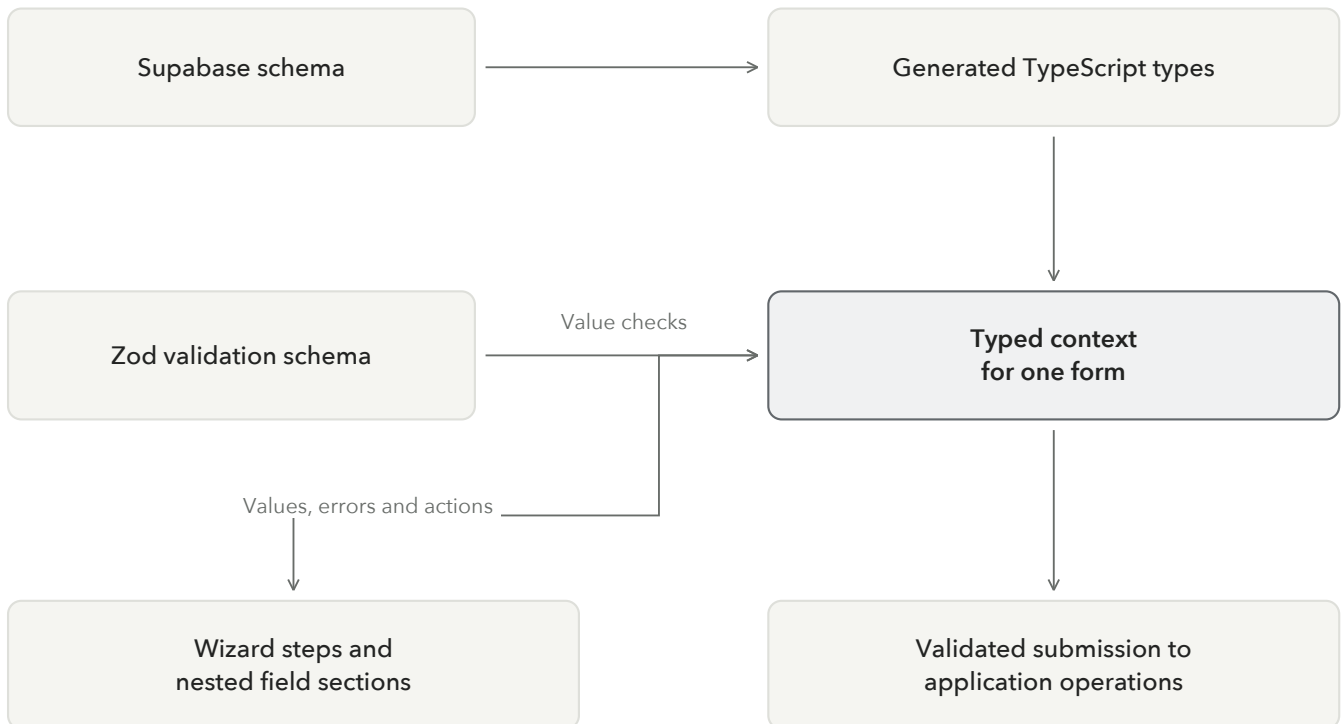
- **Generated types:** catch mismatched field names and value types during development.
- **Zod schemas:** check actual form values and provide validation feedback.
- **Access rules:** decide whether the staff member may perform the operation.

Form models also needed to account for the task being performed. An edit, a product selection and a multi-step upload could use the same database types without requiring the same fields or screen layout.

Shared forms across 15+ screens

Shared form context

Complex forms were split across steps and nested components. A shared, typed form context let those sections work with the same values, errors and actions. The hook-based form layer kept that state connected without putting the entire form into one file.



One form owns its state. Separate steps and components work with that same context.

This supported multi-step wizards and large forms whose fields appeared in several sections. A section could handle its part of the task without owning a second copy of the submission data. Local form state also preserved longer entries across page reloads.

Shared components and screen structure

Repeated field layouts, validation feedback and processing states followed shared patterns. Individual screens could focus on their business task while reusing the same form behavior.

This mattered as the back office grew beyond one upload screen. Generated types and shared form patterns helped new features fit the existing data model and staff workflow. The application also moved from Next.js 12 to the Next.js 13 App Router, with authentication and server-rendering integrations adapted around the same operations process.

Search, images and AI

Product lookup and autocomplete

External search supplied product matches and metadata for selection. Staff could reuse the matching product and colour data, then check stock without leaving the form. Search results and availability stayed separate from generated copy: the AI summary was not the source of product identity or stock information.

Image processing

The image workflow accepted a set of photos through file or Base64 input. Remove.bg handled background removal. The processing pipeline then produced WebP variants, multiple sizes and small blurred previews for progressive loading.

Processing did not block the rest of the form. Visible status let staff see which assets were ready and review the results. Automating the repeated image work was a direct part of reducing preparation time per item.

AI-generated summaries

In 2023, this became my first production use of the OpenAI API. The CMS used completed item fields, a defined example and an output format to generate a product summary. Staff reviewed the draft alongside the catalogue data and processed images before publishing.

The AI feature was part of an existing task. Search and autocomplete helped staff select and complete the item; summary generation supplied the first draft of its copy. The final form brought those outputs together for review.

Summary

The back office turned a process spread across tools into one guided workflow. Typical item preparation fell from about 15 minutes to about 3. Existing data stayed in place, while typed forms, controlled updates and API integrations made the work easier for staff to complete and the application easier to extend.